第五章：矢量数据处理进阶

谭振宇

西北大学城市与环境学院

2026 年 03 月 04 日

数据生成

属性操作

空间查询

常用处理

空间分析

空间数据库

学习目标

通过本章的学习，我们将掌握以下内容：

学习目标

1. 能够将测量数据转为空间矢量数据。
2. 对矢量数据的属性进行增删改查操作。
3. 基于 GDAL 进行空间查询。
4. 对矢量数据要素进行简单操作（如裁剪、参数融合、投影变换）。
5. 基于 GDAL 进行常见的空间分析操作（缓冲区、网络分析、叠置）。
6. 基于 GDAL 操作空间数据库。

表格数据转为矢量数据

下面展示了如何将 CSV 表格数据转为 Shapefile 数据。案例中使用了 `Pandas` 库读取数据，然后基于 GDAL 矢量数据的基本概念生成 Shapefile。

主要步骤

1. 使用 Pandas 读取 CSV 文件。
2. 使用 `ogr.GetDriverByName` 创建并初始化 `DataSource` 对象。
3. 创建 WGS84 空间参考 (`osr.SpatialReference`)。
4. 创建图层 (`Layer`) 并逐一定义并添加属性字段 (`ogr.FieldDefn`)。
5. 遍历 DataFrame 行，新建要素 (`Feature`) 并赋值，同时将其几何坐标 (`Geometry`) 以 WKT 格式创建并附加，最终完成图层保存。

示例：创建数据集对象和图层

读取数据后，建立 DataSource、空间环境，进而创建 Layer 并附加字段（表头）结构。

```python
from osgeo import ogr, osr
import pandas as pd

df = pd.read_csv("cities.csv")

# 1. 创建 DataSource 驱动与文件
ds = ogr.GetDriverByName('ESRI Shapefile').CreateDataSource("cities.shp")

# 2. 从 EPSG 代码建立 WGS84 投影
srs = osr.SpatialReference()
srs.ImportFromEPSG(4326)

# 3. 创建点状图层
layer = ds.CreateLayer('Cities', srs, ogr.wkbPoint)

# 4. 追加属性字段定义
name_fd = ogr.FieldDefn('Name', ogr.OFTString)
name_fd.SetWidth(24)
layer.CreateField(name_fd)
layer.CreateField(ogr.FieldDefn('Population', ogr.OFTReal))
layer.CreateField(ogr.FieldDefn('Country', ogr.OFTString))
```

示例：创建要素并赋值

利用 apply 函数将表格中每行信息转为要素，添加到图层中。

```python
# 新建Feature并且给其属性赋值
def add_feature(item):
    feature = ogr.Feature(layer.GetLayerDefn())
    feature.SetField('Name', item['name'])
    feature.SetField('Population', item['population'])
    feature.SetField('Country', item['country'])

    # 设置Feature的几何属性Geometry
    point = ogr.CreateGeometryFromWkt(f"POINT ({item['longitude']} {item['latitude']})")
    feature.SetGeometry(point)
    # 创建Feature
    layer.CreateFeature(feature)

# axis=1表示将函数作用于DataFrame的行
df.apply(add_feature, axis=1)
ds.FlushCache()

del ds
```

直接使用 ogr2ogr 工具

除了编写 Python 代码，还可以直接使用 GDAL 提供的命令行工具 `ogr2ogr` 将实测的表格数据转为 Shapefile 矢量数据。

```
ogr2ogr -s_srs EPSG:4326 -t_srs EPSG:4326 \
        -oo X_POSSIBLE_NAMES=LONG \
        -oo Y_POSSIBLE_NAMES=LAT \
        -mapFieldType String=Real \
        -f "ESRI Shapefile" \
        StationTemperature.shp StationTemperature.csv
```

命令参数说明

- `-s_srs` / `-t_srs`：分别指定原始和目标空间参考系统（EPSG:4326 即 WGS 84 坐标系）。
- `-oo X_POSSIBLE_NAMES=LONG`：指定 CSV 文件中可能作为 x 坐标的字段名为 `LONG`。
- `-mapFieldType String=Real`：将字段类型从字符串映射为实数类型。

数据生成

属性操作

空间查询

常用处理

空间分析

空间数据库

属性操作简介（CRUD）

对于矢量数据中每个图层中要素的属性数据，读者可以简单将其看作一个**二维表格**。学过数据库的读者，可能会想到关系数据库的概念。

目前空间矢量数据的属性数据大部分就是以关系表的形式进行存储的。而对于关系数据库的操作，常用的就是**增删改查**（Create、Retrieve、Update、Delete，即CRUD）操作。

增 (Create)

如：添加一个省简称字段 `Abbr`。

删 (Delete)

如：删除已添加的多余字段。

改 (Update)

如：给省名称统一加上“市”或“省”的后缀。

查 (Retrieve)

如：找出高中数量大于 1 万所的省份。

属性操作之增：添加简称字段

为图层增加表示简称的 Abbr 自定义属性段。核心思路是先建字典映射，再利用 CreateField，最后循环读写。

```python
# 必须设置 update=True 允许覆盖写操作
ds = ogr.Open(r"Province.shp", update=True)
layer = ds.GetLayer()
names_dict = {'北京': '京', '新疆': '新', '宁夏': '宁'} # 省略

# 添加一个字符串类型的全新字段
field = ogr.FieldDefn('Abbr', ogr.OFTString)
field.SetWidth(5)
layer.CreateField(field)

for feature in layer:
    name = feature.GetField('NAME')
    feature.SetField('Abbr', names_dict.get(name, ''))

    # !必不可少：覆盖应用修改完毕的要素让其真正刷入生效
    layer.SetFeature(feature)
```

属性操作之删：删除特定字段

西北大学

我们再尝试把该 Abbr 字段删除掉。删除的方法包括两个步骤：第一步，从属性表中找到该字段的位置索引；第二步，删除该字段。

```python
# 1. 寻找字段索引的函数
def get_field_index_by_name(layer: ogr.Layer, name: str):
    defn: ogr.FeatureDefn = layer.GetLayerDefn()
    for i in range(defn.GetFieldCount()):
        if name == defn.GetFieldDefn(i).GetName():
            return i
    raise ValueError(f'{name} not found')

# 2. 调用 DeleteField 执行删除
ds: ogr.DataSource = ogr.Open(r"Province.shp", update=True)
layer: ogr.Layer = ds.GetLayer()

index = get_field_index_by_name(layer, 'Abbr')
layer.DeleteField(index)
```

GDAL 提供的 `DeleteField()` 方法传入的参数必须是要删除字段的**索引编号**，所以需要手写一个查询器将其映射出来。

属性操作之改：更新省市名称后缀

这里要更新 NAME 字段：给直辖市名称后添加“市”，自治区后添加“自治区”，特别行政区后添加“特别行政区”，其他的添加“省”。

```python
# 填充属性值
for feature in layer:
    name: str = feature.GetField('NAME')

    # 逻辑操作：增加相应的后缀标识
    if name in ('北京', '天津', '重庆', '上海'):
        name += '市'
    elif name in ('内蒙古', '广西', '宁夏', '新疆', '西藏'):
        name += '自治区'
    elif name in ('香港', '澳门'):
        name += '特别行政区'
    else:
        name += '省'

    feature.SetField('NAME', name)
    layer.SetFeature(feature)
```

完成思路同样是遍历图层中的每一个 Feature 要素，然后通过 `SetField()` 方法更新属性值。

属性操作之查：查询方法

属性数据查询是对属性表中存储的数据进行自定义搜索的操作，通常可以通过如下两种方式进行：

使用 SQL 查询

矢量数据的属性一般都是以关系表进行保存的，所以用户可以使用关系数据库查询语言 SQL 进行数据查询。GDAL 支持部分 SQL 查询功能。

遍历要素查询

用户可以遍历图层 `Layer` 中包含的所有 `Feature` 要素，然后读取要素的属性数据运用 Python 逻辑进行筛选过滤得到自己想要的结果。

示例：基于 SQL 的查询法

使用 SQL 查询是最直白的方式，支持聚合与排序子句。

```python
ds: ogr.DataSource = ogr.Open(fn)
layer: ogr.Layer = ds.GetLayer()

# 选择出中学数量大于1万所的省份
query: str = f'SELECT NAME, HighSchool FROM {layer.GetName()} WHERE HighSchool > 10000'
selected: ogr.Layer = ds.ExecuteSQL(query)
for feature in selected:
    print(feature.GetField('NAME'))

# 选择出中学数量最多的省份（使用排序与取首个记录）
query: str = f'SELECT NAME, HighSchool FROM {layer.GetName()} ORDER BY HighSchool DESC'
selected: ogr.Layer = ds.ExecuteSQL(query)
print(f"中学数量最多的省份: {selected.GetFeature(0).GetField('NAME')}")
```

对于极值的检索，理论上可使用嵌套的 `SELECT` 与聚合 `MAX` 实现，但在 GDAL 中直接使用 `ORDER BY` 倒序选取第一条 `GetFeature(0)` 是更合理的做法，避免由于解析器嵌套导致的错误失效。图层名称中尽量不要包含中文。

示例：基于要素遍历的查询法

西北大学

如果不使用 SQL，我们可以使用原生 Python 中自带的 `filter()` 与 `sorted()` 对可迭代对象进行处理。

```python
# 使用 filter 函数对要素属性进行过滤
selected = list(filter(lambda f: f.GetField('HighSchool') > 10000, layer))
for feature in selected:
    print(feature.GetField('NAME'))

# 使用 sorted 方法对要素进行自定义排序，这里使用逆序
selected_sorted = sorted(layer, key=lambda f: f.GetField('HighSchool'), reverse=True)
print(selected_sorted[0].GetField('NAME'))
print(selected_sorted[0].GetField('HighSchool'))
```

使用第二种遍历的方式更加方便调试一些。如果对 SQL 语言较为熟悉，推荐使用 SQL 这种声明式编程的方式。

数据生成

属性操作

空间查询

常用处理

空间分析

空间数据库

空间查询概述

空间查询是根据地物的空间位置进行查询的一种数据检索方式。

OGC 简单要素规范定义了空间几何体之间的基本空间关系：`Equals` (相等), `Disjoint` (不相交), `Intersects` (相交), `Touches` (接触), `Crosses` (穿过), `Within` (在内部), `Contains` (包含), `Overlaps` (重叠)

1. **使用 SQL 语句**：使用支持空间查询的 SQL 语句。但这种方式只对特定数据源有效，某些不支持。
2. **使用空间过滤**：使用 GDAL 提供的 `SetSpatialFilter()` 方法。但这种方式主要用于选择给定外接多边形框内的地物，不能实现精确或其他类型的空间查询。
3. **读取核心 Geometry 对象 (推荐)**：读取每个要素包含的 `Geometry`，手动筛选。因 GDAL 内的 Geometry 对象几乎实现了所有 OGC 定义关系，灵活性最强。

示例：基于地理范围的空间查询

西北大学

目标：从省的面状数据中找出湖北省，然后遍历城市的点数据看是否落在湖北省境内。

```python
# 获取湖北省作为基底
lyr_province: ogr.Layer = ds_province.GetLayer()
ft_hubei = next(filter(lambda f: '湖北' in f.GetField('NAME'), lyr_province))

lyr_city = ds_city.GetLayer()
# 使用 Within() 过滤出落在湖北省境内的所有市
selected = filter(
    lambda f: f.GetGeometryRef().Within(ft_hubei.GetGeometryRef()),
    lyr_city
)

for city in selected:
    print(city.GetField('name'))
```

通过获取图层包含的要素集合，使用 Python 内置属性函数与 `Geometry` 的方法对该集合实行精准筛选。

示例：基于距离的空间查询

西北大学

目标：找出离武汉市最近的三座城市。

```python
cities = ds.GetLayer()

# 用 filter 函数找出武汉市
city: ogr.Feature = next(filter(lambda f: '武汉' in f.GetField('name'), cities))

# 调用 ResetReading() 方法特别重要，如果不重置，后面对 Feature 的遍历会出错！
cities.ResetReading()

# 根据每个市到武汉市的距离进行排序
selected = sorted(cities, key=lambda f: f.GetGeometryRef().Distance(city.GetGeometryRef()))

# 从下标1开始，因为排序后距离自身的 0 在第 0 位应当排除
for i in range(1, 4):
    print(selected[i].GetField('name'))
```

特别注意：迭代器的指针经过了拨动以后（如 next），需要进行重置 `ResetReading()` 或重新加载以作进一步检索应用。

数据生成

属性操作

空间查询

常用处理

空间分析

空间数据库

裁剪 (Clip)

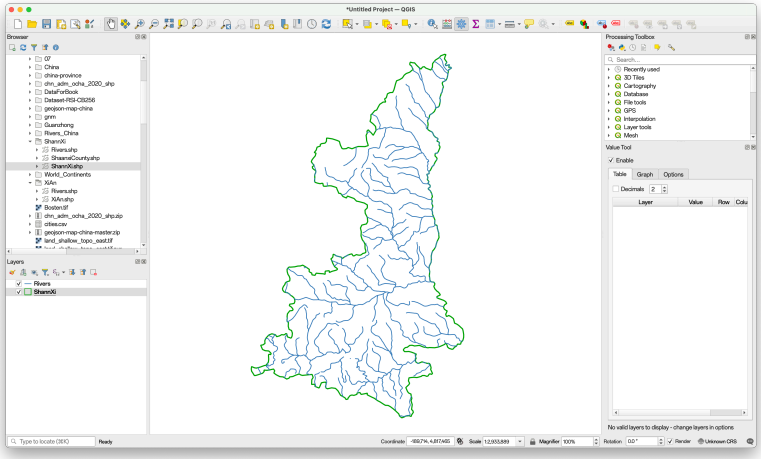
在空间分析中常使用代表研究范围的多边形边界去裁剪全局数据集。

调用 `Layer.Clip()` 即可完成干预。**注意**：裁剪双方必须具有相同投影体系！

```
# 获取原始数据层与裁剪基准层
in_lyr = ogr.Open("River.shp").GetLayer()
extent_lyr = ogr.Open("ShannXi.shp").GetLayer()

# 新建作为输出的空壳图层，保持坐标与几何种类一致
out_lyr = out_ds.CreateLayer('Clipped',
    in_lyr.GetSpatialRef(), in_lyr.GetGeomType())

# 执行基于图形范围的裁切剥离
in_lyr.Clip(extent_lyr, out_lyr)
```



陕西省内河流提取结果

投影变换 (Reproject)

投影变换是指将地理空间数据从一种坐标参考系统转换到另一种坐标参考系统的过程。在进行投影变换时需要明确变换之前数据的大地参考系和地图投影的定义。

方法 1：使用命令及高层封装实现（推荐）

使用 `ogr2ogr` 命令行工具简单准确：

```
ogr2ogr -t_srs "+proj=aea +lat_1=25 ..." China_Projected.shp China.shp
```

在 Python 中，该命令可通过 `gdal.VectorTranslate()` 执行同等转换：

```
srs_def = """+proj=aea +lat_1=25 +lat_2=47 +lat_0=30 +lon_0=105 ..."""
gdal.VectorTranslate(dst_file, src_file, dstSRS=srs_def, reproject=True)
```

投影变换

西北大学

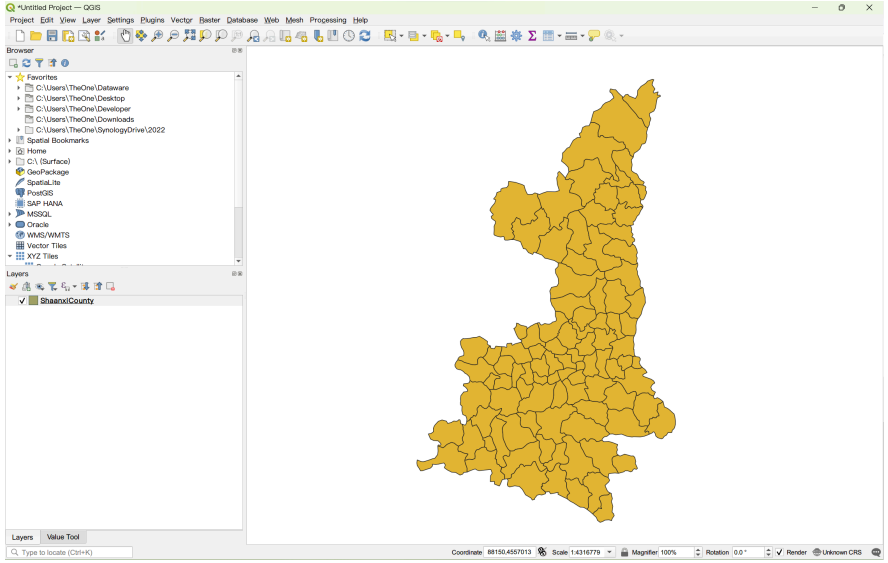
方法 2：基本 API 手工实现

1. 需要手工建立 `CreateDataSource` 与 `dst_layer`。
2. 根据源文件创建目标的每个属性字段定义。
3. 创建转换对象：`osr.CoordinateTransformation(src_srs, dst_srs)`。
4. 对每个循环源文件的 `Geometry` 应用 `Transform(ctx)`，赋予目标新变量并最后写入 `ExportToWkt` 的 prj 文件。

在不需要针对单独特定的个别元素实行干预时，请尽量使用封装的 `gdal.VectorTranslate()` 函数。

要素融合 (Dissolve)

将具有物理共界联系的多要素合并为一个集合体。调用 `Geometry.UnionCascaded()` 执行兼具拓扑检查的复杂合并。



融合县级行政区得到省级外边界

```python
# 初始化多面集装箱 wkbMultiPolygon
geoms = ogr.Geometry(ogr.wkbMultiPolygon)

for feat in in_lyr:
    feat.geometry().CloseRings() # 确立闭合形态
    geoms.AddGeometry(feat.geometry())

# 多要素合并，抹平等价拓扑冗杂相交处
union = geoms.UnionCascaded()

out_feat = ogr.Feature(defn)
out_feat.SetGeometry(union)
out_lyr.CreateFeature(out_feat)
```

数据生成

属性操作

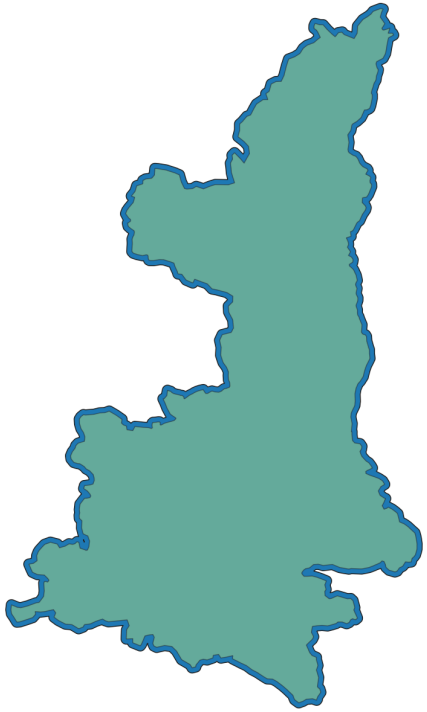
空间查询

常用处理

空间分析

空间数据库

缓冲区分析 (Buffer)



```python
# 遍历对象为每个面层发起 Buffer 操作
for feature in in_lyr:
    geometry = feature.GetGeometryRef()

    # 建立外包。此处的距离数位单位基准
    # 必定等价关联于原图层本身定义的大地网格数值
    buff = geometry.Buffer(6500)

    out_feat = ogr.Feature(def_feat)
    out_feat.SetGeometry(buff)
    out_lyr.CreateFeature(out_feat)
```

给行政边界添加晕线

叠置分析 (Overlay)

叠置分析是一种将多个地理图层进行叠加和综合处理，以揭示不同图层之间的空间关系和相互作用的方法：

联合 (Union)

合并两个图层并保留所有输入图层的几何和属性信息。

相交 (Intersection)

提取共同区域，结果仅包含两图层重叠的特区。

差异 (Difference)

从 A 图层中去除与 B 图层重叠交合所在的部分。

对称差 (Sym. Diff.)

也就是提取独立部分，保留两图层不重合的互异孤体模块。

代码实现：叠置分析

依靠 GDAL 中 Geometry 已经承载完成底层数算方法便能立即使用。

```
# 相交部分：对于两组图层可以使用循环判定相交提取输出
if geom2.Intersects(geom1):
    output.append(geom2.Intersection(geom1))

# 其他核心方法演示（单 Geometry 层级）：
# 联合求并
union = poly1.Union(poly2)

# 差异擦除
asym_diff = poly1.Difference(poly2)

# 对称相斥
sym_diff = poly1.SymDifference(poly2)
```

网络分析 (Network Analysis)

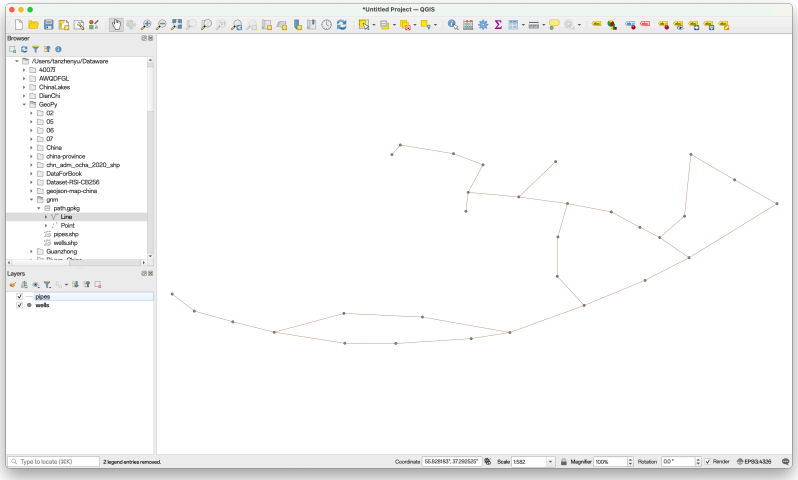
研究网络节点与边的连通性并优化路由。GDAL 提供 `osgeo.gnm` 支持点群汇聚拓扑线网并求算出可行解。

```python
from osgeo import gnm

driver = gdal.GetDriverByName('GNMFile')
ds = driver.Create('.', 0,0,0, gdal.GDT_Unknown, options=["..."])
ds.CopyLayer(lyr_pipes, 'pipes')
ds.CopyLayer(lyr_wells, 'wells')

# 转换系统并以双向可逆边权执行相近点连接
dn = gnm.CastToGenericNetwork(ds)
ret = dn.ConnectPointsByLines(['pipes', 'wells'],
    1e-5, 1, 1, gnm.GNM_EDGE_DIR_BOTH)

# 拓扑成立后即可运用 Dijkstra 求出第40至60号阀井的最短路长
result = dn.GetPath(40, 60, gnm.GATDijkstraShortestPath)
```



Dijkstra 求解最短路径

数据生成

属性操作

空间查询

常用处理

空间分析

空间数据库

空间数据库概念

空间数据库（Spatial Database）不仅仅能处理常规的二维表格数据，更能高效存储检索位置信息并管理带有索引的点、线、面实体集。

开源对象关系数据库 PostgreSQL 提供了庞健的 PostGIS 扩展模块。它遵循 OGC 规范，提供丰富的空间操作函数，使其成为最为广泛和功能强大的开源空间数据库系统。

安装流程（Ubuntu 下）：

```
sudo apt install postgresql postgis
sudo service postgresql start
sudo -u postgres psql
alter user postgres with password 'YourPassword'
```

命令行批量导入：shp2pgsql

通过 shp2pgsql 命令可以轻松将本地的 Shapefile 直接传送转换写入数据库并创建索引（如示例引入全世界地理多边形表述）。

```
# 注意中间管道串流技术 | 连结 PostgreSQL 指令接收段落
shp2pgsql -s 4326 -I "World_Continents" our_world.world_continent \
  | psql -h localhost -d postgis_in_action -U postgres -W
```

命令参数配置解释

- -s：指定输入的空间投影参考系编号。
- -I：指定在新建关系表的对应实体处预先建立空间索引，极大增加并发查检索速度表现。

通过 GDAL 连接数据库读取图层

西北大学

既然我们能够通过 Python 操作单体本地数据文件，我们也一定可借助 GDAL 标准化组件进行跨域异端挂载读取空间数据库实例。它的返回结果使用依然使用 Dataset 去操作：

```python
# 连接固定的格式配置命令环:
db_server = "127.0.0.1"
str_conn = f"PG: host={db_server} dbname={db_name} user={db_user} password={db_passwd}"

# 利用 Open 发起协议通信，连入远端!
conneciton = ogr.Open(str_conn)

# 如同抽取独立文件夹一般直接加载其中子表
name = "our_world.world_continent"
layer = conneciton.GetLayer(name)

for feat in layer:
    print(feat.GetField('continent'))

del conneciton
```

本章小结

本章作为重要的空间要素掌控环节，主要知识点总结归纳如下：

- 掌握使用 Python 进行 CSV 等格式化文档转换为矢量对象。
- 对于扩展的 SQL 查询及原生代码循环获取修改要素进行了实战使用（增删改查）。
- 对于 OGR 对象底层空间 OGC 关系统系能够利用 `Within`, `Distance` 等探底测算与利用。
- 熟练使用 GDAL 提供的跨封装对象实现处理的要素模型融合抽离（`Clip`、`UnionCascaded`）。
- 对空间计算进行系统涉猎：建立 `Buffer` 发散、重叠异差 `Overlay` 模型抽取，以及实现建立 `gnm` 拓扑管道网构建出最短连接点通路模型。
- 基本掌握基于 PostGIS 配置导入并接入现代工业生产级别空间数据库操作架构。